

Basic Structure of Java Program:

```
package your.package.name;           // Package declaration (optional)
import java.util.*;                   // Import statements (optional)
public class ClassName {               //Class Declaration (required)
    // Main method - program entry point
    public static void main(String[] args) {
        // Statements
    }
}
```

Explanation:

1.Package declaration: Specifies the package (group of related classes) in which the class is defined.

2.Import statement: Used to include predefined or user-defined Java classes from other packages.

3.Class Declaration: Every Java program must have a class. It defines the blueprint of the program.

4.Main method: This is the entry point of any Java application.

5.Statements: Actual instructions or code written inside the main method to perform tasks.

Java Feature:

1.Simple Syntax: Java syntax is very straightforward and very easy to learn. Java removes complex features like pointers and multiple inheritance, which makes it a good choice for beginners.

2.Object Oriented: Java is a pure object-oriented language. It supports core OOP concepts like, **Class, Objects, Inheritance, Encapsulation, Abstraction, Polymorphism.**

3.Platform Independent: Java is platform-independent because of Java Virtual Machine (JVM). When we write Java code, it is first compiled by the compiler and then converted into bytecode (which is platform-independent). This byte code can run on any platform which has JVM installed.

4.Interpreted: Java code is not directly executed by the computer. It is first compiled into bytecode. This byte code is then understood by the JVM. This enables Java to run on any platform without rewriting code.

5.Scalable: Java can handle both small and large-scale applications. Java provides features like multithreading and distributed computing that allows developers to manage loads more easily.

6.Portable: When we write a Java program, the code first gets converted into bytecode and this bytecode does not depend on any operating system or any specific computer. We can simply execute this bytecode on any platform with the help of JVM. Since JVMs are available on most devices and that's why we can run the same Java program on different platform.

7.Secured and Robust: Java is a reliable programming language because it can catch mistakes early while writing the code and also keeps checking for errors when the program is running. It also has a feature called exception handling that helps deal with unexpected problems smoothly.

8.Memory Management: Memory management in Java is automatically handled by the Java Virtual Machine (JVM).

- Java garbage collector reclaim memory from objects that are no longer needed.
- Memory for objects are allocated in the heap

- Method calls and local variables are stored in the stack.

9.High Performance: Java is faster than old interpreted languages. Java program is first converted into bytecode which is faster than interpreted code. It is slower than fully compiled languages like C or C++ because of interpretation and JIT compilation process. Java performance is improve with the help of Just-In-Time (JIT) compilation, which makes it faster than many interpreted languages but not as fast as fully compiled languages.

10.Multithreading: Multithreading in Java allows multiple threads to run at the same time.

- It improves CPU utilization and enhancing performance in applications that require concurrent task execution.
- Multithreading is especially important for interactive and high-performance applications, such as games and real-time systems.
- Java provides build in support for managing multiple threads. A thread is known as the smallest unit of execution within a process.

11.Rich Standard Library: Java provides various pre-built tools and libraries which is known as Java API. Java API is used to cover tasks like file handling, networking, database connectivity (JDBC), security, etc. With the help of these libraries developers save a lot of time and ready to use solutions and can also build a powerful application.

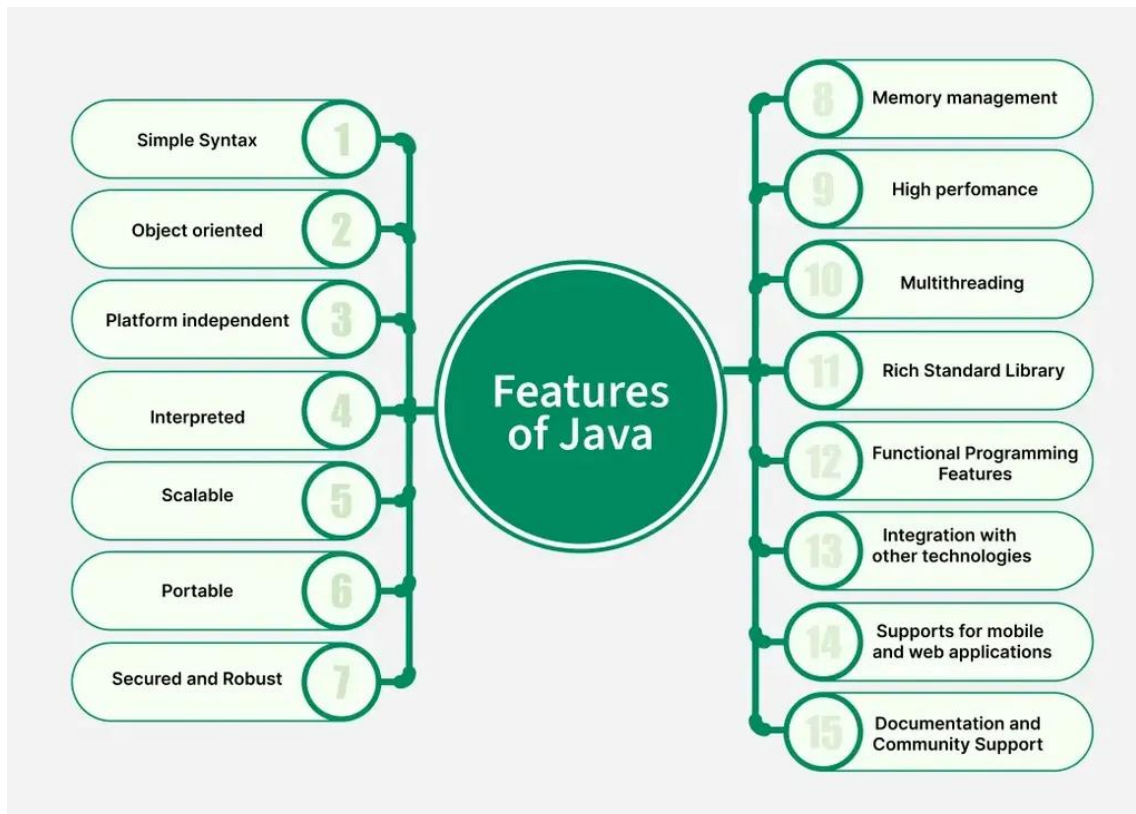
12. Functional Programming Features: Since Java 8, Java supports functional programming. **Lambda expressions** let us write short code blocks without full methods. **Stream API** helps process data easily without loops. **Functional interfaces** have one method and work well with lambdas to make code simple and reusable.

13.Integration with Other Technologies: Java can easily work with many languages and tools as well. For example, Java can connect with C and C++ with the help of Java Native Interface (JNI). Java is very popular for building websites and webservices like [RESTful & SOAP](#). In Java we can use JDBC for databse connectivity also Java is the main language for android development. As we can see Java works so well with so many different technologies that's the reason developer prefers Java more to create scalable and powerful application.

14.Support for Mobile and Web Application: Java offers support for both web and mobile applications.

- For web development: Java offers technologies like JSP and Servlets, along with frameworks like Spring and Springboot, which makes it easier to build web applications.
- For mobile development: Java is the main language for Android app development. The Android SDK uses special version of Java and its various tools to build mobile apps for Android devices.

15.Documentation and Community Support: Java provide documentation which includes guides, API references, and tutorials for easy learning. Java has a large and active global community contributing to open-source projects, and resources. This community support helps developers solve problems and stay updated with new advancements.



Java Tokens: In Java, Tokens are the smallest elements of a program that is meaningful to the compiler. They are also known as the fundamental building blocks of the program. Tokens can be classified as follows:

1.Keyword: Keywords are pre-defined or reserved words in a programming language. Each keyword is meant to perform a specific function in a program. Since keywords are referred names for a compiler, they can't be used as variable names because by doing so, we are trying to assign a new meaning to the keyword which is not allowed. **Java** language supports the following keywords: break, catch, finally, throws, return, void, int, etc.

2.Identifiers: Identifiers are the names we give to variables, functions, and arrays in a program. They are created by the user and must start with a letter or an underscore (_) but can include letters and numbers after that. Identifiers should not be the same as keywords like int, if, or while, because those are reserved by the language. Once you create an identifier, you can use it later in the program to refer to its value. There's also a special type of identifier called a label, which is used with goto statements.

3.Constants/Literals: Constants are also like normal variables. But the only difference is, their values cannot be modified by the program once they are defined. Constants refer to fixed values. They are also called as literals. Constants may belong to any of the data type.

Syntax: final data_type variable_name;

4.Operators: Java provides many types of operators which can be used according to the need. They are classified based on the functionality they provide. Some of the types are.

i.Arithmetic Operators: These operators involve the mathematical operators that can be used to perform various simple or advanced arithmetic operations on the primitive data types referred to as the operands. These operators consist of various unary and binary operators that can be applied on a single or two operands.

Operators	Result
+	Addition of two numbers
-	Subtraction of two numbers
*	Multiplication of two numbers
/	Division of two numbers
%	(Modulus Operator)Divides two numbers and returns the remainder

ii.Assignment Operators: Assignment operators are used to give values to variables. The value on the right is assigned to the variable on the left. Both must have the same data type, and the right side must be defined or a constant.

Syntax: variable operator value;

1.Simple Assignment Operator: The Simple Assignment Operator is used with the “=” sign where the left side consists of the operand and the right side consists of a value. The value of the right side must be of the same data type that has been defined on the left side.

2.Compound Assignment Operator: The Compound Operator is used where +, -, *, and / is used along with the = operator.

1.(=) operator: This is the most straightforward assignment operator, which is used to assign the value on the right to the variable on the left. This is the basic definition of an assignment operator and how it functions.

Syntax: num1 = num2;

Example:

```
a = 10;
```

```
ch = 'y';
```

2.(+=) operator: This operator is a compound of '+' and '=' operators. It operates by adding the current value of the variable on the left to the value on the right and then assigning the result to the operand on the left.

Syntax: num1 += num2;

Example:

```
a += 10
```

This means,

```
a = a + 10
```

3.(-=) operator: This operator is a compound of '-' and '=' operators. It operates by subtracting the variable's value on the right from the current value of the variable on the left and then assigning the result to the operand on the left.

Syntax: num1 -= num2;

Example:

```
a -= 10
```

This means,

```
a = a – 10
```

4.(*=) operator: This operator is a compound of '*' and '=' operators. It operates by multiplying the current value of the variable on the left to the value on the right and then assigning the result to the operand on the left.

Syntax: num1 *= num2;

Example:

```
a *= 10
```

This means,

```
a = a * 10
```

5.(/=) operator: This operator is a compound of '/' and '=' operators. It operates by dividing the current value of the variable on the left by the value on the right and then assigning the quotient to the operand on the left.

Syntax: num1 /= num2;

Example:

```
a /= 10
```

This means,

```
a = a / 10
```

6.(%=) operator: This operator is a compound of '%' and '=' operators. It operates by dividing the current value of the variable on the left by the value on the right and then assigning the remainder to the operand on the left.

Syntax:

```
num1 %= num2;
```

Example:

```
a %= 3
```

This means,

```
a = a % 3
```

iii.Logical Operators: Logical operators are used to perform logical "AND", "OR", and "NOT" operations, i.e., the functions similar to AND gate and OR gate in digital electronics. They are used to combine two or more conditions/constraints or to complement the evaluation of the original condition under particular consideration.

AND Operator (&&): If(a && b) [if true execute else don't].

OR Operator (||): If(a || b) [if one of them is true to execute; else don't].

NOT Operator (!): !(a<b) [returns false if a is smaller than b].

iv.Relational Operator: Java Relational Operators are a bunch of binary operators used to check for relations between two operands, including equality, greater than, less than, etc. They return a boolean result after the comparison and are extensively used in looping statements as well as conditional if-else statements and so on.

Operator 1: 'Equal to' operator (==) This operator is used to check whether the two given operands are equal or not. The operator returns true if the operand at the left-hand side is equal to the right-hand side, else false.

Syntax: var1 == var2;

Operator 2: 'Not equal to' Operator(!=) This operator is used to check whether the two given operands are equal or not. It functions opposite to that of the equal-to-operator. It returns true if the operand at the left-hand side is not equal to the right-hand side, else false.

Syntax: var1 != var2;

Operator 3: 'Greater than' operator(>) This checks whether the first operand is greater than the second operand or not. The operator returns true when the operand at the left-hand side is greater than the right-hand side.

Syntax: var1 > var2;

Operator 4: 'Less than' Operator(<) This checks whether the first operand is less than the second operand or not. The operator returns true when the operand at the left-hand side is less than the right-hand side. It functions opposite to that of the greater-than operator.

Syntax: var1 < var2;

Operator 5: Greater than or equal to (>=) This checks whether the first operand is greater than or equal to the second operand or not. The operator returns true when the operand at the left-hand side is greater than or equal to the right-hand side.

Syntax: var1 >= var2;

Operator 6: Less than or equal to (<=) This checks whether the first operand is less than or equal to the second operand or not. The operator returns true when the operand at the left-hand side is less than or equal to the right-hand side. **Syntax:** var1 <= var2

v.Unary Operators: Java unary operators are the types that need only one operand to perform any operation like increment, decrement, negation, etc. It consists of various arithmetic, logical and other operators that operate on a single operand.

Operator 1: Unary minus(-) This operator can be used to convert a positive value to a negative one.

Syntax: -(operand);

Illustration: a = -10;

Operator 2: 'NOT' Operator(!) This is used to convert true to false or vice versa. Basically, it reverses the logical state of an operand.

Syntax: !(operand);

Illustration: cond = !true; *// cond < false*

Operator 3: Increment(++)

It is used to increment the value of an integer. It can be used in two separate ways:

3.1: Post-increment operator: When placed after the variable name, the value of the operand is incremented but the previous value is retained temporarily until the execution of this statement and it gets updated before the execution of the next statement.

Syntax: num++;

Illustration:

```
num = 5;  
num++ = 6;
```

3.2: Pre-increment operator: When placed before the variable name, the operand's value is incremented instantly.

Syntax: ++num;

Illustration:

```
num = 5;  
++num = 6;
```

Operator 4: Decrement (--)

It is used to decrement the value of an integer. It can be used in two separate ways:

4.1: Post-decrement operator: When placed after the variable name, the value of the operand is decremented but the previous values is retained temporarily until the execution of this statement and it gets updated before the execution of the next statement.

Syntax: num--;

Illustration:

```
num = 5;  
num-- = 4; // Value will be decremented before execution of next statement.
```

4.2: Pre-decrement operator: When placed before the variable name, the operand's value is decremented instantly.

Syntax: --num;

Illustration:

```
num = 5;
```

```
--num = 4; //output is 5, value is decremented before execution of next statement
```

Operator 5: Bitwise Complement(~) This unary operator returns the one's complement representation of the input value or operand, i.e, with all bits inverted, which means it makes every 0 to 1, and every 1 to 0.

Syntax: ~(operand);

String Manipulation: Strings are used for storing text. A String variable contains a collection of characters surrounded by double quotes.

1.length(): A String in Java is actually an object, which contain methods that can perform certain operations on strings.

Ex:

```
String txt = "Atomic";
```

```
System.out.println("The length of the txt string is: " + txt.length());
```

2.indexOf(): The indexOf() method returns the index (the position) of the first occurrence of a specified text in a string (including whitespace).

Ex:

```
String txt = "Please locate where 'locate' occurs!";
```

```
System.out.println(txt.indexOf("locate")); // Outputs 7
```

3.Concatenation: The + operator can be used between strings to combine them. This is called concatenation.

Ex:

```
String firstName = "John";
```

```
String lastName = "Doe";
```

```
System.out.println(firstName + " " + lastName);
```

4.toUpperCase() and toLowerCase(): The toUpperCase() method converts a string to upper case letters. The toLowerCase() method converts a string to lower case letters.

Ex:

```
String txt = "Hello World";
```

```
System.out.println(txt.toUpperCase());
```

```
System.out.println(txt.toLowerCase());
```

5.substring(): The substring() method returns a substring from the string. If the *end* argument is not specified then the substring will end at the end of the string.

Ex:

```
String myStr = "Hello, World!";
```

```
System.out.println(myStr.substring(7, 12));
```

6.replace(): The replace() method searches a string for a specified character, and returns a new string where the specified character(s) are replaced.

Ex:

```
String myStr = "Hello";
```

```
System.out.println(myStr.replace('l', 'p'));
```

7.lastIndexOf(): The lastIndexOf() method returns the position of the last occurrence of specified character(s) in a string.

Ex:

```
String myStr = "Hello planet earth.";
System.out.println(myStr.lastIndexOf("planet"));
```

8.equal(): The equals() method compares two strings, and returns true if the strings are equal, and false if not.

Ex:

```
String myStr1 = "Hello";
String myStr2 = "Hello";
System.out.println(myStr1.equals(myStr2));
```

9.equalsIgnoreCase(): The equalsIgnoreCase() method compares two strings, ignoring lower case and upper case differences. This method returns true if the strings are equal, and false if not.

Ex:

```
String myStr1 = "Hello";
String myStr2 = "HELLO";
System.out.println(myStr1.equalsIgnoreCase(myStr2));
```

10.compareTo(): The compareTo() method compares two strings lexicographically. The comparison is based on the Unicode value of each character in the strings. Use compareToIgnoreCase() to compare two strings lexicographically, ignoring lower case and upper case differences.

Use the equals() method to compare two strings without consideration of Unicode values.

Ex:

```
String myStr1 = "Hello";
String myStr2 = "Hello";
System.out.println(myStr1.compareTo(myStr2));
```

Exception Handling: Exception handling in Java is a way to handle errors that occur while the program is running. Instead of crashing the program, Java lets you catch the error and take action.

Error: Errors are serious problems that happen outside the control of the program. They usually cannot be handled or fixed by the program. Examples: OutOfMemoryError, StackOverflowError. These are in the java.lang.Error class.

Exception: Exceptions are issues that occur due to code mistakes or user input. These can be handled using try-catch blocks. Examples: ArithmeticException, NullPointerException, IOException. These are in the java.lang.Exception class.

try and catch: The try statement allows you to define a block of code to be tested for errors while it is being executed. The catch statement allows you to define a block of code to be executed, if an error occurs in the try block. The try and catch keywords come in pairs.

Syntax:

```
try {  
    // Block of code to try  
}  
catch(Exception e) {  
    // Block of code to handle errors  
}
```

Ex:

```
public class Main {  
    public static void main(String[] args) {  
        try {  
            int[] myNumbers = {1, 2, 3};  
            System.out.println(myNumbers[10]);  
        } catch (Exception e) {  
            System.out.println("Something went wrong.");  
        }  
    }  
}
```

OUTPUT: Something went wrong.

finally: The finally statement lets you execute code, after try...catch, regardless of the result.

Ex:

```
public class Main {  
    public static void main(String[] args) {  
        try {  
            int[] myNumbers = {1, 2, 3};  
            System.out.println(myNumbers[10]);  
        } catch (Exception e) {  
            System.out.println("Something went wrong.");  
        } finally {  
            System.out.println("The 'try catch' is finished.");  
        }  
    }  
}
```

OUTPUT: Something went wrong.

The 'try catch' is finished.

throw: The throw statement allows you to create a custom error. The throw statement is used together with an exception type. There are many exception types available in Java: ArithmeticException, FileNotFoundException, ArrayIndexOutOfBoundsException, SecurityException, etc

Ex:

```
public class Main {  
    static void checkAge(int age) {  
        if (age < 18) {  
            throw new ArithmeticException("Access denied - You must be at least 18 years old.");  
        }  
    }  
}
```

```
    else {  
        System.out.println("Access granted - You are old enough!");  
    }  
}  
  
public static void main(String[] args) {  
    checkAge(15);                // Set age to 15 (which is below 18...)  
}  
}
```

OUTPUT:

Exception in thread "main" java.lang.ArithmeticException: Access denied - You must be at least 18 years old.

at Main.checkAge(Main.java:4)
at Main.main(Main.java:12)

File Handling:**File Operations:**

1.Create a File: In order to create a file in Java, you can use the `createNewFile()` method. If the file is successfully created, it will return a Boolean value `true` and `false` if the file already exists.

Ex:

```
import java.io.File;  
import java.io.IOException;  
public class CreateFile  
{  
    public static void main(String[] args)  
    {  
        try {  
            File Obj = new File("myfile.txt");  
            if (Obj.createNewFile()) {  
                System.out.println("File created: " + Obj.getName());  
            }  
            else {  
                System.out.println("File already exists.");  
            }  
        }  
        catch (IOException e) {  
            System.out.println("An error has occurred.");  
            e.printStackTrace();  
        }  
    }  
}
```

2.Write to a File: We use the `FileWriter` class along with its `write()` method in order to write some text to the file.

Ex:

```
import java.io.FileWriter;  
import java.io.IOException;  
public class WriteFile  
{  
    public static void main(String[] args)  
    {
```

```
try {
    FileWriter Writer = new FileWriter("myfile.txt");
    Writer.write("Files in Java are seriously good!!");
    Writer.close();
    System.out.println("Successfully written.");
}
catch (IOException e) {
    System.out.println("An error has occurred.");
    e.printStackTrace();}}
```

3.Read from a File: We will use the Scanner class in order to read contents from a file.

Ex:

```
import java.io.File;
import java.io.FileNotFoundException;
import java.util.Scanner;
public class ReadFile
{
    public static void main(String[] args)
    {
        try {
            File Obj = new File("myfile.txt");
            Scanner Reader = new Scanner(Obj);

            while (Reader.hasNextLine()) {
                String data = Reader.nextLine();
                System.out.println(data);
            }
            Reader.close();
        }
        catch (FileNotFoundException e) {
            System.out.println("An error has occurred.");
            e.printStackTrace();
        }
    }
}
```

4.Delete a File: We use the delete() method in order to delete a file.

Ex:

```
import java.io.File;
public class DeleteFile
{
    public static void main(String[] args)
    {
        File Obj = new File("myfile.txt");
        if (Obj.delete()) {
            System.out.println("The deleted file is : " + Obj.getName());
        }
        else {
            System.out.println(
                "Failed in deleting the file.");
        }
    }
}
```

```
}}}
```

All combined:

Ex:

```
import java.io.*;
import java.util.Scanner;
public class FileOperationsOneTry {
    public static void main(String[] args) {
        try {
            File file = new File("example.txt");
            // Create the file
            if (file.createNewFile()) {
                System.out.println("File created: " + file.getName());
            } else {
                System.out.println("File already exists.");
            }
            // Write to the file
            FileWriter writer = new FileWriter(file);
            writer.write("This is written in one try-catch block.\nJava is powerful!");
            writer.close();
            System.out.println("Data written successfully.");
            // Read from the file
            Scanner sc = new Scanner(file);
            System.out.println("\nFile content:");
            while (sc.hasNextLine()) {
                System.out.println(sc.nextLine());
            }
            sc.close();
            // Delete the file
            if (file.delete()) {
                System.out.println("\nFile deleted successfully.");
            } else {
                System.out.println("Failed to delete the file.");
            }
        } catch (IOException | FileNotFoundException e) {
            System.out.println("An error occurred: " + e.getMessage());
        }
    }
}
```

JVM:

1.JVM (Java Virtual Machine) runs Java applications as a run-time engine. JVM is the one that calls the main method present in a Java code. Execute line by line hence, known as interpreter.

2.JVM is a part of JRE (Java Runtime Environment). Java applications are called WORA (Write Once Run Anywhere). This means a programmer can develop Java code on one system and expect it to run on any other Java-enabled system without any adjustments. This is all possible because of the JVM.

3. When we compile a .java file, .class files (containing byte-code) with the same class names present in the .java file are generated by the Java compiler. This .class file goes through various steps when we run it. These steps together describe the whole JVM.

JRE:

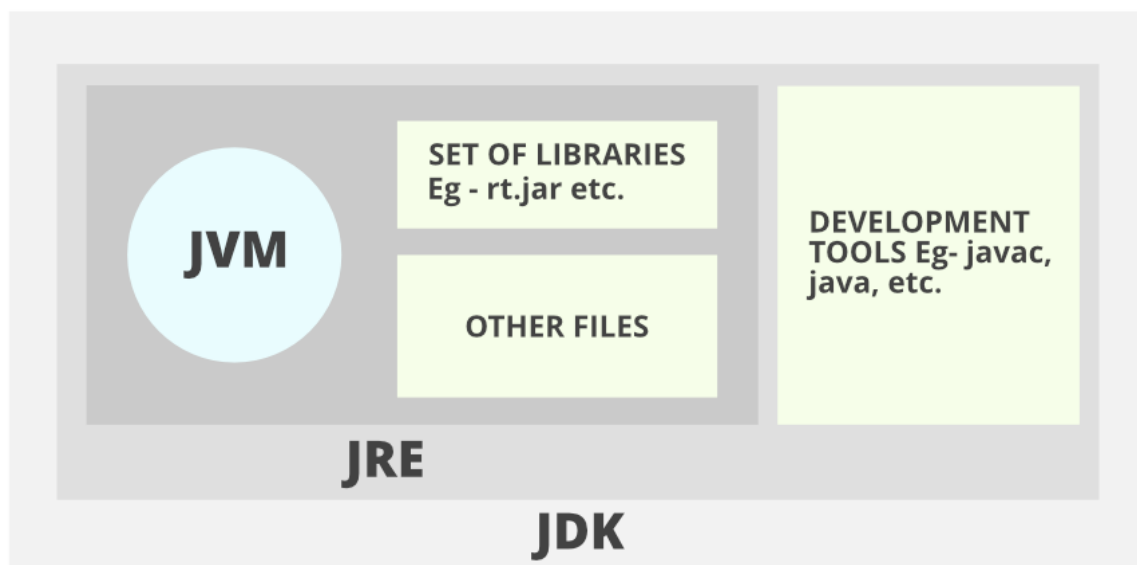
1. Java Runtime Environment (JRE) is an open-access software distribution that has a Java class library, specific tools, and a separate JVM. In Java, JRE is one of the interrelated components in the Java Development Kit (JDK).

2. It is the most common environment available on devices for running Java programs. Java source code is compiled and converted to Java bytecode.

3. If you want to run this bytecode on any platform, you need JRE. The JRE loads classes, checks memory access, and gets system resources. JRE acts as a software layer on top of the operating system.

JDK:

The Java Development Kit (JDK) is a cross-platformed software development environment that offers a collection of tools and libraries necessary for developing Java-based software applications and applets. It is a core package used in Java, along with the JVM (Java Virtual Machine) and the JRE (Java Runtime Environment).



Abstraction: Abstraction in Java is the process of hiding the implementation details and only showing the essential details or features to the user. It allows to focus on what an object does rather than how it does it. The unnecessary details are not displayed to the user.

Key features of abstraction:

- Abstraction hides the complex details and shows only essential features.
- Abstract classes may have methods without implementation and must be implemented by subclasses.
- By abstracting functionality, changes in the implementation do not affect the code that depends on the abstraction.

Implementation: Java provides two ways to implement abstraction:

1. Abstract Classes (Partial Abstraction).
2. Interface (100% Abstraction).

Ex:

WAP to demonstrate abstraction in java.

CODE:

```
abstract class Vehicle{
    abstract void start();
    void fuel(){
        System.out.println("Refueling the vehicle.");
    }
}
class Car extends Vehicle{
    void start(){
        System.out.println("Starting the Vehicle.");
    }
}
public class Abs{
    public static void main(String args[]){
        Car c = new Car();
        c.start();
        c.fuel();
    }
}
```

Packages: A package in Java is a mechanism for organizing related classes and interfaces into a single unit. It provides a way to group classes and interfaces based on their functionality making it easier to manage and maintain a large codebase.

Ex:

Step1:

```
package mypack; // Package declaration
public class Message {
    public void show() {
        System.out.println("Hello from the package!");
    }
}
```

Step 2:

```
import mypack.Message; // Import the class from the package
public class Main {
    public static void main(String[] args) {
        Message msg = new Message();
        msg.show();
    }
}
```

Step3: Compile:

```
javac mypack/Message.java
javac Main.java
```

Step 4: Run:

```
java Main
```

Interface: An interface in Java is an abstract type that defines a contract or a set of behaviors that a class must implement. It specifies method signatures (names, return types, and parameters) but does not provide implementations. The Interfaces are used to achieve abstraction and support multiple inheritances in Java.

Ex:

```
interface Animal {  
    void sound();           // method in interface  
}  
  
class Dog implements Animal{  
    public void sound(){  
        System.out.println("Dog barks");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Dog d = new Dog();  
        d.sound();  
    }  
}
```

Difference Between Package:

Characteristics	Package	Interface
Purpose	Organize related classes and interfaces into a single unit.	Define a contract for classes to implement.
Syntax	Declared using package keyword.	Declared using interface keyword.
Usage	Group classes and interfaces based on functionality.	Define method signatures that classes implementing the interface must provide implementations.
Accessibility	Packages can have access modifiers to control visibility.	Interface methods are implicitly public and abstract.
Multiple Inheritance	Does not support multiple inheritance.	Supports multiple inheritance as a class can implement multiple interfaces.
Example	package mypackage;	public interface Printable { void print(); }

History of Java/Jakarta:

1.1991 Creation Started: Java was started by James Gosling and his team at Sun Microsystems. It was originally called Oak, designed for embedded devices like TVs and set-top boxes.

2.1995 Renamed to Java: Oak was renamed Java (named after Java coffee) and launched for internet applications because of its ability to run on any platform. It promised **Write Once, Run Anywhere** (WORA), providing no-cost run-times on popular platforms.

3.1996 First Version Released: Java 1.0 was officially released, allowing developers to create web-based and desktop applications.

4.2006 GPL: Sun released much of Java as free and open source software under the terms of the GNU General Public License (GPL).

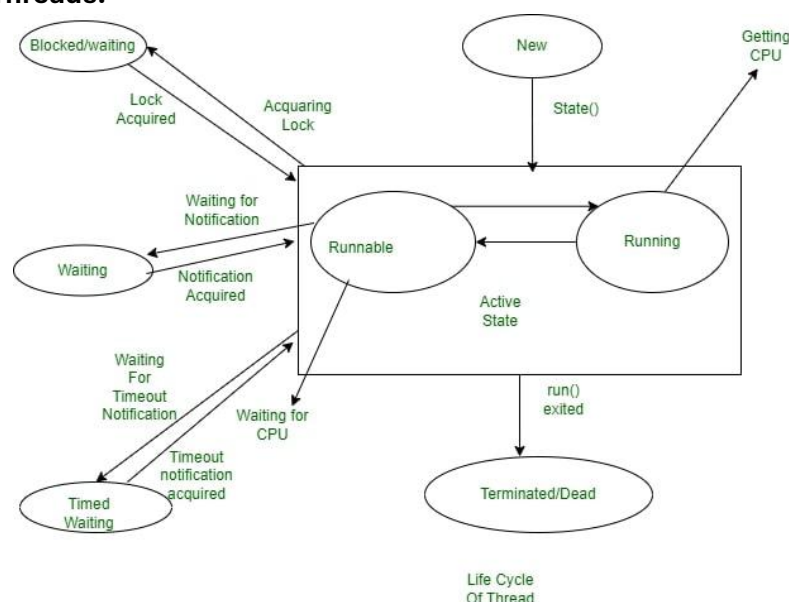
5.2007: Sun finished the process, making all of Java's core code free and open-source, aside from a small portion of code to which Sun did not hold the copyright.

6.2009 Oracle Acquired Sun Microsystems: Oracle Corporation took over Java after buying Sun Microsystems.

7..2014 and beyond Regular Updates: Java introduced features like Lambda expressions, Streams, and modules in Java 8, and continued evolving with versions 11, 17, and the latest versions (every 6 months).

Threads: Java threads are lightweight subprocesses, representing the smallest unit of execution with separate paths. The main advantage of multiple threads is efficiency (allowing multiple things at the same time). For example, in MS Word, one thread automatically formats the document while another thread is taking user input. Additionally, multithreading ensures quick response, as other threads can continue execution even if one gets stuck, keeping the application responsive.

Life Cycle of Threads:



1.New State: By default, a thread will be in a new state, in this state, code has not yet started execution.

2.Active State: When a thread is created and the `start()` method is called, it enters the Active state. This state includes two sub-states:

- **Runnable State:** The thread is ready to run but is waiting for CPU time. The thread scheduler decides which thread runs next. Since multiple threads share CPU time, each thread runs for a short time and then waits again.
- **Running State:** When the scheduler gives CPU to the thread, it enters the Running state. Once its time slot ends, it goes back to the Runnable state to wait for its next turn.

3.Waiting/Blocked State: If a thread can't continue right now but may later, it enters either the waiting or blocked state:

- **Waiting:** For example, if Thread T1 needs the camera but T2 is using it, T1 waits until T2 is done.
- **Blocked:** If two threads (like T2 and T3) try to access the same resource at the same time, one may get blocked until the resource is free.

The thread scheduler manages these threads, clearing low-priority ones and giving CPU time to others.

4.Timed Waiting State: To avoid a thread waiting too long (starvation), Java uses the Timed Waiting state. For example, if T1 takes too long doing a critical task, T2 may starve. To fix this, methods like `sleep()` let a thread wait only for a set time. Once the time is over, the thread becomes active again.

5.Terminated State: A thread will be in terminated state, due to the below reasons:

- Termination is achieved by a thread when it finishes its task normally.
- Sometimes threads may be terminated due to unusual events like segmentation faults, exceptions...etc. and such kind of termination can be called abnormal termination.
- A terminated thread means it is dead and no longer available.

Java:- `wait()`, `notify()` and `notifyAll()` methods:

1.`wait()`: The `wait()` method makes a thread voluntarily give up its lock on an object, allowing another thread to execute code within a synchronized block. The thread that calls `wait()` will enter a waiting state until another thread calls `notify()` or `notifyAll()` on the same object, allowing it to resume execution.

2.`notify()`: The `notify()` method wakes up one of the waiting threads on the same object. If multiple threads wait, it is not specified which one will be awakened. The awakened thread will then compete for the lock on the object.

3.`notifyAll()`: The `notifyAll()` method wakes up all waiting threads on the same object. This can be useful when multiple threads are waiting, and you want them to be notified simultaneously.

Difference between them:

Sr. No.	Aspect	notify()	notifyAll()
1	Notification Behavior	Sends a signal to only one thread that is waiting for the lock (monitor).	Sends a signal to all waiting threads, waking them up at the same time.
2	Thread Selection	You cannot control or know which thread will be notified — it's chosen by the JVM.	All threads are notified, but only one thread will acquire the lock at a time; others will go back to waiting.
3	Risk Factor	Higher risk: If the chosen thread doesn't proceed (e.g., due to a condition mismatch), the others stay waiting — leading to possible deadlock or program stuck.	Lower risk: Even if one thread can't proceed, others might — improving chances that progress will happen.
4	Performance	More efficient in terms of CPU and memory because only one thread wakes up and competes for the lock.	Less efficient: All threads wake up, compete for the lock, then most go back to waiting — which uses more resources.
5	When to Use	Use when only one specific thread needs to proceed and the task is not interchangeable.	Use when any thread can continue the task (i.e., threads are interchangeable), or when you're unsure which thread will meet the required condition.

Program to create thread: Thread can be referred to as a lightweight process. Thread uses fewer resources to create and exist in the process; thread shares process resources. The main thread of Java is the thread that is started when the program starts. The slave thread is created as a result of the main thread. This is the last thread to complete the execution.

1.Thread Class: The Thread class provides constructors and methods for creating and operating on threads. The thread extends the Object and implements the Runnable interface. It starts a newly created thread. Thread moves from new state to runnable state when it gets a chance, executes the target run() method.

Method: *public void start()*

// Extending Thread in Class

import java.io.*;

// Class inheriting Thread class

class MyThread extends Thread

{

 // Overriding the run method

 @Override

 public void run(){

 System.out.print("Welcome");

 }

}

class G

```
{
    public static void main(String[] args)
    {
        // Creating thread
        MyThread t1 = new MyThread();
        // Starting thread
        t1.start();
    }
}
```

2. Runnable Interface: Any class with instances that are intended to be executed by a thread should implement the Runnable interface. The Runnable interface has only one method, which is called run().

```
import java.io.*;
class MyRunnable implements Runnable{
    public void run(){
        System.out.println("Thread is running.");
    }
}
public class G{
    public static void main(String args[]){
        MyRunnable r = new MyRunnable();
        Thread t = new Thread(r);
        t.start();
    }
}
```

3. Using Lambda Expressions: Lambda Expressions are very useful for creating the Threads in the cases when the operations to be performed are limited.

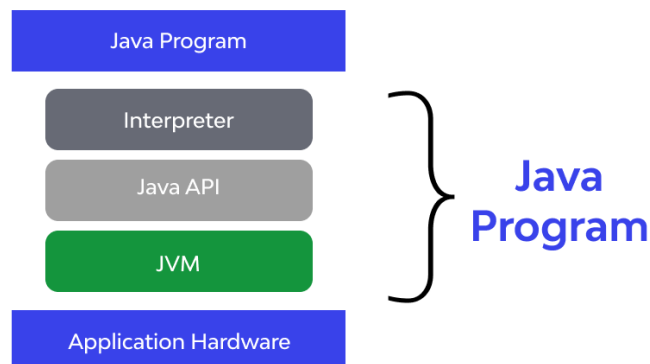
// Using Lambda Method to Create Thread

```
import java.io.*;
class G
{
    public static void main (String[] args)
    {
        // Lambda Thread Created
        Thread thread1 = new Thread(() -> {
            // Operations Performed for thread1
            System.out.println("Lambda Thread running");
        });
        // Running the Thread
        thread1.start();
    }
}
```

4. Using ExecutorService (for Managing Thread Pools): The **ExecutorService** framework provides a higher-level way to manage threads. It allows you to create a pool of threads and manage their execution.

```
// Using ExecutorService
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
public class G
{
    public static void main(String[] args)
    {
        // Create a thread pool with 2 threads
        ExecutorService threads = Executors.newFixedThreadPool(2);
        // Submitting Threads as Runnable
        threads.submit(() -> {
            System.out.println("Task 1 is running in " + Thread.currentThread().getName());
        });
        threads.submit(() -> {
            System.out.println("Task 2 is running in " + Thread.currentThread().getName());
        });
        // Shutdown the threads/executor
        threads.shutdown();
    }
}
```

Java API: In Java, an API (Application Programming Interface) is a set of ready-made tools like classes and methods that help developers build programs. Instead of writing all the code yourself, you can use this pre-written code to make your work easier and faster. It helps different parts of a program talk to each other and saves time by reusing existing code.



In Java, the API comes with the Java Development Kit (JDK), which includes three main parts:

1. **Java Compiler:** Turns your written code into a simpler form called byte-code that the computer can understand.
2. **Java Virtual Machine (JVM):** Runs the byte-code and shows the result of your program.
3. **Java API:** A collection of built-in tools and features (like classes and methods) that help you write programs faster without starting from scratch.

The Java API lets you use ready-made parts to build your program, saving time and making development easier. Developers can use these tools to quickly access useful features and complete their work faster.

Importance:

1. For example, imagine an online shopping app. When users browse products, add items to their cart, make payments, or track their orders, everything works smoothly behind the scenes. This is possible because the app uses Java APIs to connect with different services like payment gateways, inventory systems, and delivery tracking. These APIs help the app work faster, more efficiently, and provide a better experience to the user without the developer writing every feature from scratch.
2. Use of API in Java grants access to a wide range of SQL support.
3. Java API brings customization of high quality accessible for developers.

Methods of Threads:

1.start(): Starts a new thread and calls its run() method.

Syntax:

```
Thread t = new Thread();
```

```
t.start(); // starts the thread
```

2.run(): Contains the code that executes when the thread runs.

Syntax:

```
class MyThread extends Thread {  
    public void run() {  
        System.out.println("Thread is running");  
    }  
}
```

3.sleep(long millis): Pauses the current thread for the specified time (milliseconds).

Syntax:

```
Thread.sleep(1000); // pause for 1 second
```

4.join(): Waits for a thread to finish before continuing execution.

Syntax:

```
t.join(); // main thread waits until t finishes
```

5.yield(): Suggests the current thread pause and let other threads execute.

Syntax:

```
Thread.yield(); // gives chance to other threads
```

6.setPriority(int p): Sets the priority of a thread (1 = MIN, 5 = NORM, 10 = MAX).

Syntax:

```
t.setPriority(Thread.MAX_PRIORITY); // 10
```

7.getPriority(): Returns the priority of the thread.

Syntax:

```
int p = t.getPriority();  
System.out.println(p);
```

8.isAlive(): Checks if the thread is still running. Returns true if alive.

Syntax:

```
boolean status = t.isAlive();
```

```
System.out.println(status);
```

9.getName(): Returns the name of the thread.

Syntax:

```
String name = t.getName();
```

```
System.out.println(name);
```

10.setName(String name): Sets a custom name for the thread.

Syntax:

```
t.setName("MyThread");
```

11.currentThread(): Returns a reference to the currently executing thread.

Syntax:

```
Thread t = Thread.currentThread();
```

```
System.out.println(t.getName());
```

12.interrupt(): Interrupts a thread, usually to stop sleeping or waiting.

Syntax:

```
t.interrupt(); // interrupts thread t
```

